

Reasoning-Driven Retrieval

Incorporating reasoning to improve retrieval

From Static to Agentic RAG

Reason → Retrieve → Reason → Retrieve

INTRODUCTION

How do we move from systems that *retrieve once and answer confidently* to systems that *reason about what they need, retrieve iteratively, and know when they have enough*?

To build up on basic RAG pipelines to systems which reason before retrieving, we need

- Retrieval driven by reasoning, not the raw prompt
- Multiple retrievals per task, shaped by prior results
- Explicit retrieval state across hops
- Termination by sufficiency, not by hop count
- Evidence selection as a separate stage from retrieval

RAG PATTERNS

We usually see three retrieval patterns commonly labelled into RAG: static, rewritten, and agentic



Static RAG

A fixed pipeline: embed the query, top-k vector search, pass chunks to the LLM.
Retrieval is a single call. There is no second chance.
The LLM sees whatever was returned, relevant or not.



Rewritten RAG

A static pipeline with one preprocessing step: the LLM rewrites the query first.
Still one retrieval call. Still one answer pass.
Better recall on bad queries, but it is not reasoning about retrieval.



Agentic RAG

The LLM decides when to retrieve, what to search for, and when to stop.
Retrieval is a tool call inside a reasoning loop.
The system observes results and adapts its next query.

PROPERTIES OF AGENTIC RAG

01

Reasoning Before Retrieval

The system reasons about what it needs to find before issuing any query. Retrieval is driven by a hypothesis, not by the raw user prompt.

02

Iterative Retrieval

The system can retrieve multiple times in one task. Each retrieval is shaped by what the previous one returned: reason → retrieve → reason → retrieve again.

03

Retrieval State

The system remembers what it has already found and what it still needs. It does not re-search the same ground.

04

Termination Criterion

The system knows when it has enough evidence to answer and stops. No fixed turn count required.

HOW THESE PROPERTIES ARE ENFORCED

Every property maps to an explicit piece of the architecture

○ Reasoning Before Retrieval

- A planner node reads state and produces the next query.
- No direct path from user prompt to retriever. If there is, we get static RAG again.

○ Iterative Retrieval

- A while-loop driven by a termination check, not by a fixed hop count.
- Each hop updates state before the next plan is formed.

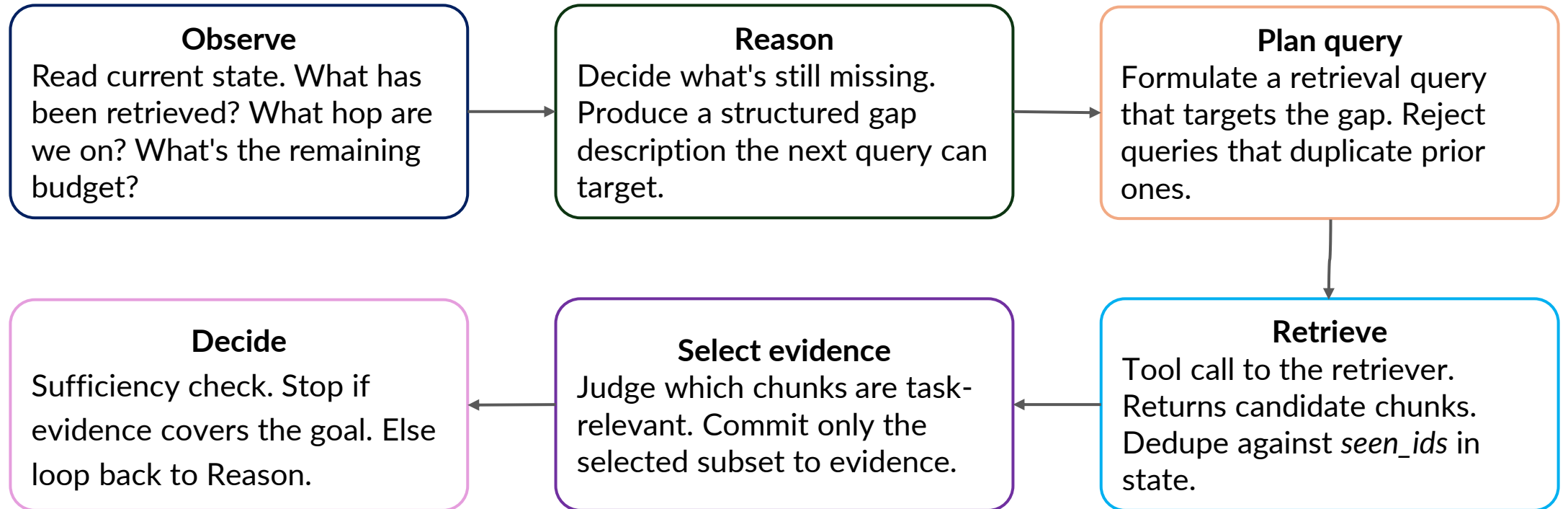
○ Retrieval State

- A structured state object: *goal, hops_used / budget, queries_issued, chunks_seen, evidence_selected, gap*
- Mutated explicitly per hop.

○ Termination Criterion

- A *sufficiency_check(state)* style function returns True when the gap is covered or when the budget is exhausted
- Explicit stop condition, never implicit.

THE LOOP



In general, an agentic RAG flow produces:

- **Answer text:** the final grounded response
- **Evidence trail:** chunk ids the answer is written from
- **Hop trace:** queries issued, results, decisions
- **Cost breakdown:** tokens per hop, total wall time
- **Termination reason:** sufficient / budget / error

EXAMPLE COMPARISON

Task: *"From our engineering-RFC corpus, summarise how we decided on CRDTs vs OT for collaborative editing."*

Static RAG (non-agentic)

1. Embed the task as-is.
2. Top-k search returns 6 chunks, mostly about CRDTs in general.
3. LLM writes a generic CRDT summary.

Problems:

- One query, never refined
- No check if the RFC is even in the returned chunks
- No memory of what was found
- Cannot recover if retrieval misses the decision thread
- Stops after one pass regardless

Agentic approach

1. **Plan:** I need the decision, the rejected alternative, and the rationale.
2. **Search**('CRDT vs OT RFC') → finds RFC-042 but no rationale section.
3. **Reason:** rationale usually sits in the 'Trade-offs' section. Retrieve that.
4. **Search**('RFC-042 trade-offs') → finds the rationale block.
5. **Reason:** did we discuss OT's specific failures? Need one more pass.
6. **Search**('OT convergence bug 2024') → finds linked incident notes.
7. **Check:** evidence covers decision, alternative, rationale → sufficient.

Result: adaptive, goal-directed, grounded

A HARDER TASK

Task: *"Was the root cause of ticket #4823 also present in any other recent support ticket or the latest changelog?"*

Agentic trace: 3 hops, 3 sources; grounded answer

1. **Plan:** I need the ticket's root cause, then related tickets, then the changelog.
2. **Retrieve**('ticket 4823 root cause') → finds the ticket + cause: 'state desync on tab reload'.
3. **Reason:** search for that cause across recent tickets.
4. **Retrieve**('tab reload state desync', last_90_days=True) → finds tickets #4791 and #4802.
5. **Reason:** check changelog for any fix or mention of this behaviour.
6. **Retrieve**('state desync', source='changelog') → finds v2.14.3 note: 'partial fix for tab reload'.
7. **Check:** evidence covers ticket #4823 + 2 related + 1 changelog entry → sufficient.

Result: "Yes, also seen in #4791 and #4802. A partial fix shipped in v2.14.3 but did not cover this case."

CHUNKS AND EVIDENCE



Chunk

- A unit of text returned by a retriever
- Examples:
 - `top_k[0]` → a 500-token paragraph
 - `vector_search(q)` → scored snippets
 - `bm25_retriever(q)` → lexical matches
- A chunk is raw material, what the retriever gave you



Evidence

- A selected, trusted, task-relevant subset of chunks that the agent commits to using
- Examples:
 - 'RFC-042 §3 trade-offs paragraph'
 - 'incident note: OT convergence bug'
 - 'benchmark table from evaluation doc'
- Evidence is what the agent decides to reason over. Chunks are the candidates.

*Static RAG dumps all chunks into the prompt
Agentic RAG selects evidence from them*

HOPS AND STEPS

Hop

- One retrieval call inside the loop
- Examples:
 - *search('CRDT vs OT')*
 - *search('RFC-042 trade-offs')*
 - *bm25('OT convergence bug')*
- Hop count = number of retrievals
Bound by *max_hops*

Step

- One full cycle of observe → reason → decide → act
- Examples:
 - Step 1: plan + first retrieval
 - Step 2: reformulate + second retrieval
 - Step 3: plan + stop (no retrieval)
- A step can plan-and-stop without retrieving
Bound by *max_steps*

Add budgets for both. A bug that loops reasoning without retrieving won't be caught by a hop cap alone.

FAILURES AND GUARDRAILS

What Fails in Practice

- **Hop + step budget**
 - Cap both hop count and step count. Default *max_hops*=5, *max_steps*=8.
 - A runaway loop cannot drain the retriever quota or exhaust the LLM budget.
- **Evidence-sufficiency check**
 - Before continuing, compare the current gap against prior hops.
 - Block redundant retrievals that ask for what the agent already has.
- **Reformulation distance guard**
 - A new query must differ from prior queries by a minimum token distance.
 - Stops loops that keep rewriting queries into near-duplicates.
- **Cost circuit breaker**
 - Track cumulative token spend per task. Emit a warning at 75% of budget; hard-fail at 100%.
 - Log the hop trace when it trips.

FAILURES AND GUARDRAILS

Guardrails Agentic RAG Needs

- **Query drift**
 - Each reformulation walks further from the original intent.
 - Fix: anchor every reformulation to the original question in the prompt.
- **Hallucinated gaps**
 - Agent decides evidence is insufficient when it actually is.
 - Fix: structured sufficiency check with an explicit coverage list, not free-form reasoning.
- **Evidence dumping**
 - Agent pastes every retrieved chunk into the prompt.
 - Fix: separate retrieval from selection; commit only the subset that survived the selection stage.
- **Silent over-retrieval**
 - Agent runs to the hop cap every time.
 - Fix: log hop-count distributions across a test set and tune the sufficiency threshold.

AGENTIC RAG SYSTEM - CHECKLIST

- ✓ Hop budget AND step budget explicitly configured
- ✓ Termination criterion is a function, not a fixed hop count
- ✓ Evidence selection stage separate from retrieval
- ✓ Reformulation distance guard in place
- ✓ Per-task cost tracker emitting tokens + latency metrics
- ✓ Hop trace logged and searchable
- ✓ Fallback path to static RAG on loop failure
- ✓ Evaluation run on a multi-hop benchmark before deploy
- ✓ 'When NOT to go agentic' documented for the team
- ✓ Oncall runbook covers 'stuck loop' and 'budget hit' failures

Model Context Protocol

*A JSON-RPC standard that lets any agent use any tool,
resource, or prompt.*

WHY WE NEED A PROTOCOL?

Instead of writing a custom adaptor for every tool of an agent, can we expose one contract that any framework, any language, and any vendor can follow?

- Agents invoke tools; tools provide local access to the agent process
- Each integration in the pipeline adds its own schema, authorisation, and transport. So, memory and resources accumulate
- There is no common contract between different frameworks; a tool written for LangGraph will not work with LlamaIndex

MCP standardises this contract

It defines tools schemas, tool calling, resources, and prompts in a fixed format that works across clients and servers

WHAT IS MCP?

- LangGraph, LlamaIndex, CrewAI are frameworks
- They run the agent loop: reason, call, observe
- MCP sits below them, as the tool wire format
- A framework can consume MCP servers as tools

Agent Frameworks

- An open JSON-RPC protocol between an MCP client and an MCP server
- Defines *list_tools*, *call_tool*, *list_resources*, *list_prompts*
- Transport-agnostic: *stdio*, Streamable HTTP
- Framework-agnostic and language-agnostic

MCP

MCP is a protocol, not an agent framework

MCP PROPERTIES

Some properties of MCP are

1. Client / Server Split

An MCP client lives inside the host application. An MCP server is a separate process. They speak JSON-RPC over a chosen transport.

2. Runtime Discovery

The client calls *list_tools*, *list_resources*, *list_prompts* at handshake. Nothing about tool names or schemas is hardcoded on the client.

3. Three Primitives

The protocol exposes exactly three kinds of capabilities: tools (actions), resources (read-only data), prompts (reusable templates).

4. Transport-Agnostic Wire

The same server source runs over stdio, SSE, or Streamable HTTP. The client changes only the transport field in its config.

MCP PROPERTIES

Some properties of MCP are

5. Composability

Any MCP client works with any MCP server. A server written today is usable three framework generations from now.

6. Reusability

Write a capability once. Any number of agents across any number of teams can consume it without per-agent glue.

7. Process Isolation

Each server runs as its own subprocess. A crashing tool cannot take down the agent. Credentials stay server-side.

8. Capabilities Handshake

On connect the server declares which primitives it supports tools, resources, prompts, sampling, roots via capabilities negotiation.

MCP PROPERTIES - EXAMPLE

Approach	Discovery	Reusability	Isolation	Composability
Native LangChain @tool	X	X	X	X
OpenAI function-call (inline)	~	~	X	X
Custom gRPC / HTTP per tool	~	~	✓	X
MCP server (stdio)	✓	✓	✓	✓
MCP server (SSE / HTTP)	✓	✓	✓	✓

PRACTICES TO AVOID

01

MCP-wrapping every @tool

Replacing all in-process functions with MCP servers because it feels more 'standard'. Pure overhead for tools that never cross a team boundary.

02

Treating MCP as an agent framework

MCP does not orchestrate. It does not plan. It is a transport plus schema. The agent loop still needs LangGraph or equivalent around it.

03

Running untrusted servers

An MCP server is a subprocess you are giving inputs to. Run unknown binaries at your own risk. Validate, pin, and sandbox.

04

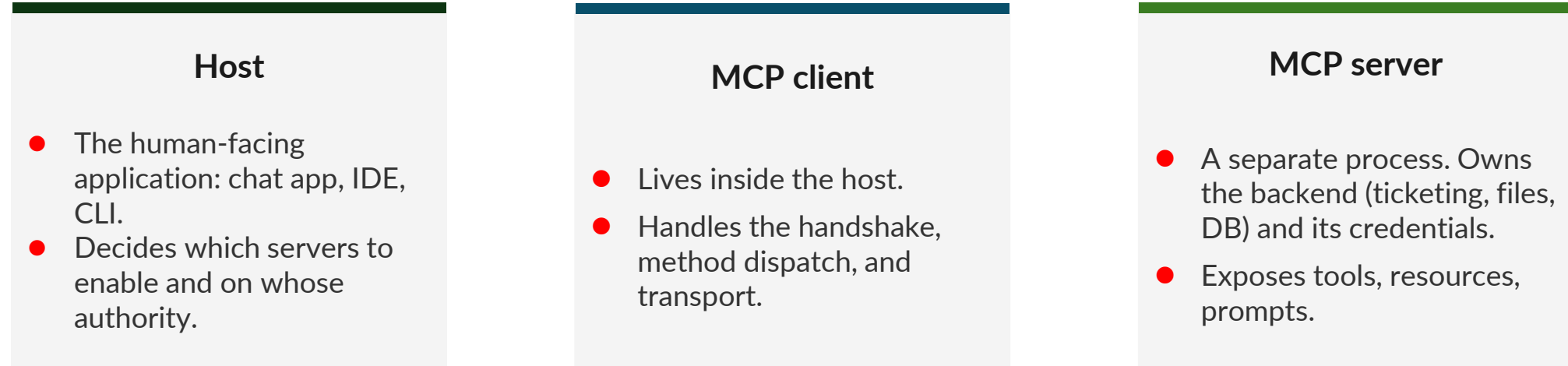
Hot-swapping tools mid-session

Tool discovery happens at handshake. Adding or removing tools while a client is running does nothing until the next reconnect.

ARCHITECTURE

We have three participants –

The host embeds the client. The client speaks MCP. The server owns the backend.



Lifecycle in four steps:

1. Host spawns the server (stdio) or opens the SSE / HTTP connection; transport is chosen per server.
2. Handshake: capabilities negotiation, then *list_tools*, *list_resources*, *list_prompts* over JSON-RPC.
3. Agent loop issues *call_tool(name, args)* for actions and *read_resource(uri)* for context.
4. Server returns JSON payloads; client routes them back to the agent as *ToolMessages*.

MCP PRIMITIVES – TOOLS AND RESOURCES

Tools

- Named functions with JSON schemas
- Calling a tool has side effects or returns fresh data
- Examples:
 - *list_tools* handshake enumerates them
 - *call_tool('create_ticket', {...})*
 - *search_kb, list_tickets, semantic_kb_search*
- Tools are for actions
- The client invokes them by name via *call_tool*.

Resources

- Read-only data exposed under URIs
- No side effects, no arguments; like HTTP GET inside MCP
- Examples:
 - project files, wiki pages
 - log snippets, configuration blobs
 - enumerated via *list_resources*; fetched with *read_resource*
- Resources are for context
- The host can cache them and show the user what data is attached

MCP PRIMITIVES – PROMPTS AND SAMPLING

Prompts

- Reusable prompt templates the server publishes for the client to surface
- Examples:
 - *review-this-pr(diff)* returns a filled instruction
 - *list_prompts* / *get_prompt* RPCs
 - server ships prompt engineering as an asset
- Prompts let the backend author, who knows the domain, own the best wording for its tools

Sampling

- A server-initiated request back to the client asking it to run an LLM completion
- Examples:
 - *summarise-document(text, max_tokens)* returns a concise summary generated by the client model
 - *classify-intent(query, labels)* returns the predicted label using the client-side model
- Sampling flips the direction; the server can ask the client's LLM for help. Powerful and rare

GUARDRAILS

MCP servers are untrusted by default Every server is a process you launch. Treat it like any other internet-facing endpoint.

1 **Validate the server binary**

Pin versions, verify checksums, do not run untrusted servers as your own user.
Classic subprocess hygiene applies.

2 **Authenticate remote transports**

stdio has no auth by design. For SSE and Streamable HTTP use OAuth + bearer tokens and check them inside each tool body.

3 **Defend against tool poisoning**

A malicious server can return tool descriptions or results that try to steer your agent. Pin descriptions at handshake; sandbox server permissions.

4 **Scope roots explicitly**

The roots primitive declares which URIs and filesystem paths a server may access.
Constrain roots per server, not per tool.

INTEGRATION

langchain-mcp-adapters introspects MCP schemas and wraps each tool as a LangChain *BaseTool*

01

Configure

MultiServerMCPClient({'support':
{command, args, transport: 'stdio'}, 'kb':
{...}}).

02

Discover

await mcp_client.get_tools(): spawns
subprocesses, runs *list_tools*, wraps
each as *BaseTool*.

03

Bind

agent = create_react_agent(llm, tools):
the agent sees MCP tools as ordinary
LangChain tools.

04

Invoke

LLM emits *tool_call*; framework routes
to MCP client; client issues *call_tool*;
ToolMessage returns.

PITFALLS

Four pitfalls to watch in production MCP deployments

01

***stdio* blocks on long tools**

The stdio transport is synchronous over one pipe.
Any tool that exceeds a few seconds should run under SSE or HTTP, or wrap blocking work in async with timeouts.

02

Tool list is cached at connect

Discovery happens once at handshake.
If the server adds a tool mid-session, the client will not see it until it reconnects.
Plan deploy windows.

03

Process management is your job

Each MCP server is a process to monitor, restart, upgrade, and observe.
Health checks, structured logs, metrics, rolling upgrades; none of it is free.

04

Latency budget still matters

Each MCP call adds ~5–50 ms of protocol overhead vs a native @tool.
Negligible for most agents; measurable for high-throughput sub-100 ms SLAs.

MCP SERVER SHIPPING - CHECKLIST

- **Capabilities:** declare exactly which primitives (tools / resources / prompts / sampling / roots) the server supports.
- **Schemas:** every `@mcp.tool()` has type hints and a docstring so `list_tools` returns a clean schema.
- **Transport:** stdio for local, SSE / Streamable HTTP for remote. Pick one per deployment and document it.
- **Auth:** for SSE / HTTP, validate bearer tokens inside every tool body, not just at the edge.
- **Roots:** constrain the URI and filesystem roots the server may touch.
- **Process isolation:** server holds credentials; host never does.
- **Error surface:** tools return structured errors, not opaque tracebacks.
- **Observability:** structured logs, metrics, health check, restart policy.
- **Reuse test:** at least one consumer outside your own team is planned or already live.

Agent Design Patterns

RECALL

Till now, we have designed agent loops of the following patterns

reason_retrieve_loop

Static RAG baseline (single-shot)

query_reformulation_multihop

Iterative pattern with query rewriting between hops. An agent loop with a step budget.

memory_selection

Four-node planner-executor: plan, route, execute, synthesise

memory_long_term_types

Structured-output plan dispatched across three memory types. Planning as a first-class artifact.

memory_short_vs_long

Summarisation over the agent's own history. A light form of reflection / reflexive behaviour.

evidence_selection

LLM reranker used structurally as a critic over retrieval candidates.

Let's try to formally define agentic patterns now

AGENT DESIGN PATTERNS

A pattern specifies three things

Node roles

What kind of work each node does. Generation, retrieval, planning, critique, verification, reflection. The role, not the exact prompt.

Edge structure

Which nodes feed which. Which edges are conditional on state. Linear, cyclic, branching, fan-out, fan-in. The topology, not the framework.

Termination condition

When the graph exits. A single-shot graph terminates after one node.

An iterative graph terminates when a predicate fires on state.

A planner-executor terminates after synthesise.

A reflexive agent may terminate in one run but modify its behaviour for next.

AGENT DESIGN PATTERNS

Pattern	Node roles	Cyclic edges	Termination	Composes
single-shot	✓	X	✓	✓
iterative	✓	✓	✓	✓
planner-executor	✓	X	✓	✓
critic	✓	~	✓	✓
verifier	✓	X	✓	✓
reflexive	✓	✓	✓	✓

✓ = defining trait ~ = optional (in critic-with-revise loop) X = absent

Every pattern composes with every other. Composition is the point.

CRITIC AND VERIFIER

Critic

- A subjective quality scorer. Returns score, critique, or rewrite suggestion.
- Examples:
 - 0-10 polish score with rewrite
 - LLM reranker over candidates
 - Code-review-style critique pass
- Critic = art teacher. 'This could be better, here is how.'

Verifier

- An objective constraint checker. Returns pass / fail against a formal spec.
- Examples:
 - JSON-schema validation
 - Does this SQL actually execute
 - Grounding check against retrieved evidence
- Verifier = security. 'Pass or do not pass.'

Subjective → critic
Objective → verifier

REFLEXIVE

Reflexive (cross-run)

- The agent writes lessons to persistent memory that later runs read from.
- Examples:
 - Reflexion-style self-lessons
 - Auto-rewritten system prompts
 - Failure-mode summary memory
- State at run $t+1$ depends on the agent's reflection at run t .

Reflection (in-run)

- The agent critiques its own current output and may revise it within the same run.
- Examples:
 - Self-correction mid-answer
 - Chain-of-thought re-check
 - Tool-call result verification
- State does not cross runs - reflection ends when the answer ships.

*Reflexive agents and reflection are closely related and frequently confused.
The difference is whether state crosses the run boundary.*

Reflection is a move inside one answer. Reflexive is a policy across many. Different contracts, different memory.

GENERATOR + CRITIC + VERIFIER - COMPOSED CASCADE

Task: *"Draft an SQL migration script that passes schema checks and reads cleanly."*

Static RAG (non-agentic)

1. Plan: write migration, then polish.
2. Generator emits a first draft.
3. Critic scores style 8/10, suggests renames.
4. Generator rewrites for readability.

Problems:

- Schema violations never checked
- Critic drifts on subjective calls
- Ships polished-but-broken SQL

Composed cascade

1. **Plan:** draft, critique, verify, then ship.
2. Generator produces a draft migration.
3. **Reason:** style needs work before schema check.
4. Critic suggests naming / structure rewrites.
5. **Plan:** now run the verifier.
6. Verifier runs dry-run against the real schema.
7. **Check:** schema pass + style $\geq$ threshold $\rightarrow$ accept.

Result: polished AND correct. Never substitute one for the other.

ARCHITECTURE

To read the agent graph and identify the pattern, apply this procedure

Count the real nodes

One real node → single-shot. A cycle → iterative. A linear multi-stage pipeline with a plan artifact → planner-executor. A generate-then-judge pair → critic or verifier. A node reading and writing long-lived memory → reflexive.

Find the conditional edges

Iterative loops have exactly one conditional edge controlling termination. Planner-executors may have conditional edges inside route for dispatch. Critic loops have a conditional edge on the score threshold. Verifiers have a conditional edge on pass-or-fail.

Classify the nodes by role

Generation, retrieval, planning, critique, verification, reflection. Two nodes doing the same kind of work separated by a conditional edge is a loop. A node reading another's output and writing a judgement is critic or verifier.

INTEGRATION

Real agents almost never implement one pattern in isolation.

Some ways the six patterns can be composed in production:

01

Planner wrapping an iterative executor

Planner decides the shape of the work once. Each sub-task in execute is itself a small iterative agent that retrieves and reasons until done.

02

Generator + critic + verifier cascade

Generator drafts. Critic improves subjective polish via a score-and-rewrite loop. Verifier gates the final output against a hard spec.

03

Iterative inner loop with verifier

Each iteration of an iterative agent exits either on its own signal or on a verifier saying the draft already satisfies the spec. Prevents the iteration cap from firing when the task was already done.

04

Reflexive layer over a planner-executor

Planner-executor handles the immediate task. A reflexive layer writes lessons across sessions so later runs start with priors.

GUARDRAILS

Each rule removes one class of silent failure mode:

01

Step budget on every loop

Iterative and reflexive loops must cap max_iterations at 3-5.
LLM-driven agent loops will occasionally decide they need just one more step forever.

02

Plan is a first-class artifact

Planner-executor plans are structured (Pydantic), logged, replayable, and auditable.

03

Critic threshold must be explicit

A critic with no score cutoff drifts forever.
The outer loop exits when score $\geq$ threshold or max revisions hit, whichever comes first.

04

Verifier is pass/fail

Verifiers gate; they do not rewrite.
If your verifier is suggesting changes, it has become a critic and is no longer enforcing a spec.

PATTERN SELECTION LADDER - CHECKLIST

1. **Q1** Can this be answered in one LLM call with available context?
Yes → single-shot. Ship it. No further patterns needed.
2. **Q2** Does the work have known sub-task structure visible before starting?
Yes → planner-executor.
No → iterative with a step budget.
3. **Q3** Does the output need quality polish across subjective dimensions?
Yes → add a critic on top of generation.
4. **Q4** Does the output need to satisfy a hard spec (schema / policy / grounding)?
Yes → add a verifier as the final gate.
5. **Q5** Should the agent learn across runs from its own past mistakes?
Yes → add reflexive memory backed by an eval.